

Smart contract audit Cyclo

Content

Project description	3
Executive summary	3
Reviews	3
Technical analysis and findings	6
Security findings	7
**** Critical	7
*** High	7
** Medium	7
* Low	7
Informational	7
General Risks	8
Approach and methodology	9



Project description

This repository implements a decentralized financial system that leverages receipt-based vaults to manage tokenized assets. It provides mechanisms for minting ERC20 shares and ERC1155 receipt NFTs, enabling transparent tracking of deposits and withdrawals. The system supports dynamic share-to-asset ratios, off-chain asset integration, and price oracle-based calculations. It incorporates features like certification, asset confiscation, and fine-grained access control to ensure compliance and trust. The design emphasizes flexibility for issuers to maintain asset pegs and profitability while offering robust auditability and user protections.

Executive summary

Type	Vault
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/cyclofinance/cyclo.sol https://github.com/rainlanguage/ethgild

Reviews

Date	Repository	Commit
18/12/25	cyclo	63d11553006bf24421c033b19f6dab0ba1bc4573
	ethgild	d2f99564935db209bf0a8267940228176dbd0303

Scope

Contracts
cyclo/src/concrete/receipt/CycloReceipt.sol
cyclo/src/concrete/vault/CycloVault.sol
cyclo/src/lib/LibCycloProdAssets.sol



cyclo/src/lib/LibCycloProdCloneFactory.sol
cyclo/src/lib/LibCycloProdOracle.sol
cyclo/src/lib/LibCycloProdReceipt.sol
cyclo/src/lib/LibCycloProdVault.sol
cyclo/script/Deploy.sol
ethgild/src/abstract/PriceOracleV2.sol
ethgild/src/abstract/ReceiptVault.sol
ethgild/src/concrete/oracle/FtsoV2LTSFeedOracle.sol
ethgild/src/concrete/oracle/PythOracle.sol
ethgild/src/concrete/oracle/SceptreStakedFlrOracle.sol
ethgild/src/concrete/oracle/TwoPriceOracleV2.sol
ethgild/src/concrete/receipt/ERC20PriceOracleReceipt.sol
ethgild/src/concrete/receipt/Receipt.sol
ethgild/src/concrete/vault/ERC20PriceOracleReceiptVault.sol
ethgild/src/concrete/vault/OffchainAssetReceiptVault.sol
ethgild/script/Deploy.sol
rain.factory/src/concrete/CloneFactory.sol
rain.math.fixedpoint/lib/LibFixedPointDecimalArithmeticOpenZeppelin.sol
rain.math.fixedpoint/lib/format/LibFixedPointDecimalFormat.sol
rain.math.fixedpoint/src/lib/FixedPointDecimalConstants.sol
rain.flare/lib/lts/LibFtsoV2LTS.sol
rain.flare/lib/sflr/LibSceptreStakedFlare.sol
rain.flare/src/err/ErrFtso.sol
rain.flare/src/lib/registry/LibFlareContractRegistry.sol
rain.intorastring/src/lib/LibIntOrAString.sol

Technical analysis and findings

Critical	0
High	0
Medium	0
Low	0
Informational	0



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

No Medium severity issue found.

* Low

No Low severity issue found.

Informational

No Informational severity issue found.



General Risks

- The `receiptInformation()` function in the Receipt contract can be invoked at any time by any user. However, there is no specific event type associated with this function. This can lead to potential confusion when reading and interpreting events, as it may not be clear which function invocation triggered a particular event.

Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.

